Stage 2 Build a capable coding harness

Building our own Claude Code/Codex harness

Extend your Stage 1 application so it handles a feature across modules, keeps context bounded, and saves work safely. This is the second of three working deliveries of your semester project.

A capable coding harness finds relevant code, carries task history through an edit–test–fix loop, and lets a user stop and reopen work. Keep your existing tools, permissions, execution limits, and bug-fix workflow working.

Your second working version

A user selects an allowed project and submits a feature request through a CLI or UI. Your controller selects context, edits code, uses failed checks to guide fixes, and saves progress. The user can cancel, reopen the session, and review the diff and check results.

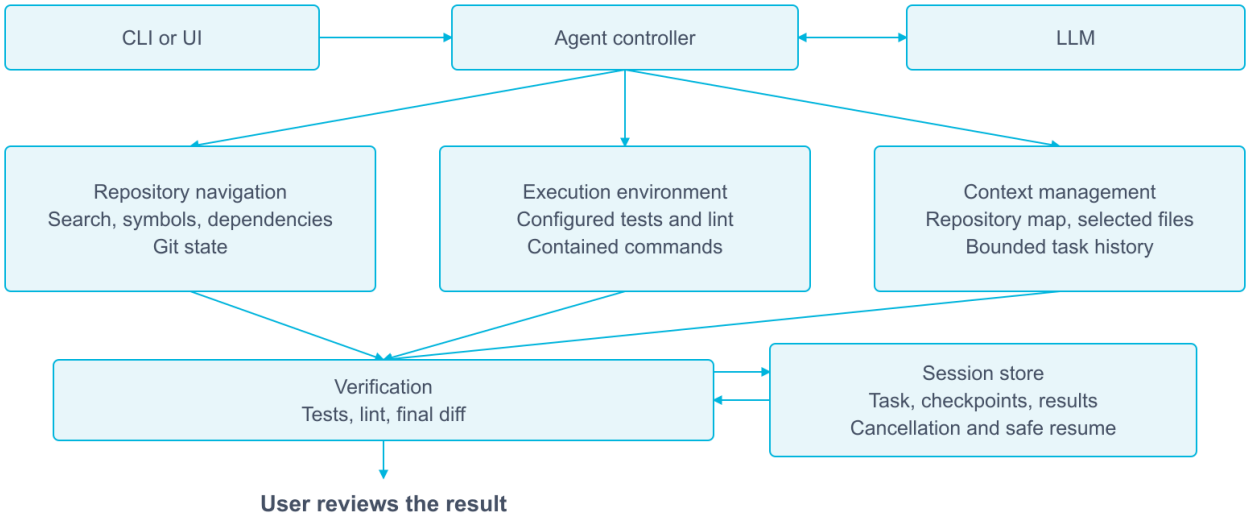


Figure 1 Stage 2 adds repository navigation, bounded context, and saved sessions.

Choose your implementation

Build your own coding harness in Python. You may use an agent framework and libraries, and reuse your lab code. You design, build, and test the application. Do not start from an existing coding harness. A clear CLI can earn full credit; a UI is also welcome.

Use any model provider you choose. Qwen 3.8 with local Ollama is one suggested option (ollama run qwen3.8). A paid service is not required.

What to build

Part	Required behavior
Navigation	Find relevant modules, symbols, and dependencies. Inspect Git state. Simple language-aware tools are enough.
Context	Build a compact repository map, select files, and shorten history within a documented size limit. Preserve trusted instructions, scope, and acceptance criteria. Keep repository text and tool output untrusted.
Task execution	Save the request, starting commit, allowed changes, checks, and task steps. Run an edit–test–fix loop with bounded retries and a clear stopping rule.
Sessions	Save and reopen work, show progress, cancel an active run, and resume from a safe checkpoint. Mark uncertain interrupted actions explicitly.
Verification	Show changed files, the final diff, test and lint results, and unfinished work. Keep failed or unavailable checks visible. Add build checks if needed.
CLI or UI	Select an allowed project, start and inspect a task, cancel, resume, and review the result. Keep interface code separate from the core logic.

Keep responsibilities clear

The example below shows one possible structure. A framework may provide several of these parts. Use classes, functions, or both; you do not need to copy these names.

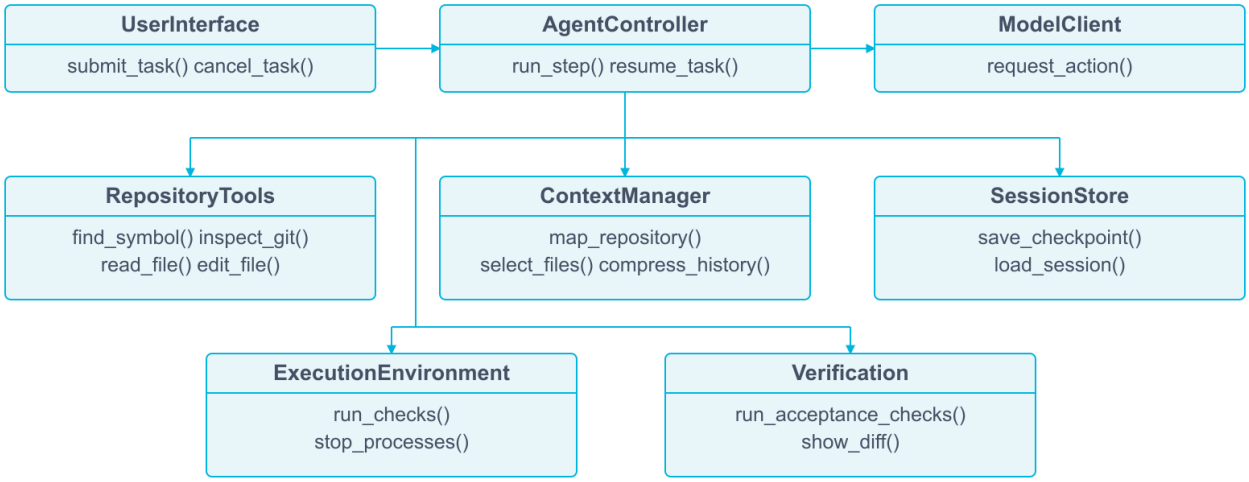


Figure 2 Example class responsibilities. Arrows mean that one component uses another.

Choose your own context and checkpoint/resume design. Explain it in the README and test it. On resume, preserve consumed limits and revalidate permissions and the saved working-copy state, including the harness's own edits. Pause on a mismatch; never replay an uncertain interrupted action automatically.

Make the loop work

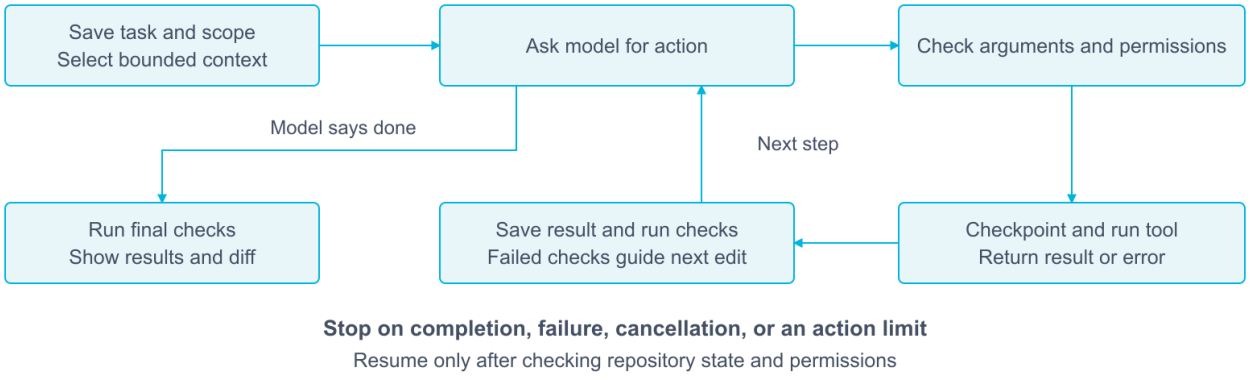


Figure 3 Failed checks guide fixes. Checkpoints support cancellation and safe resume.

Protect the working environment

Use a disposable copy of the target repository. Keep credentials and unrelated host files outside executed code. Run repository code and tests inside a contained environment; an existing sandbox setup is allowed. A Git worktree alone is not a sandbox.

Check tool arguments and permissions in application code. Treat repository text and model replies as untrusted input. Keep push, merge, and deployment disabled. A stuck command must be stoppable without leaving child processes or further writes behind.

Tests to include

Test	Expected result
Navigation	Find the modules, symbols, and dependencies needed for a task. Show the expected Git state.
Context	Select relevant files and enforce the size limit. Compression preserves task constraints and keeps untrusted text separate.
Session storage	Reopen saved task details, steps, history, errors, and check results after the application restarts.
Cancellation	Test one representative cancellation during a model request or command. Stop further actions or writes and clean up any child processes.
Safe resume	Resume against the saved working-copy state, including the harness's edits. Recheck permissions and limits; pause on mismatches or uncertain actions.
Checks and loop	Verify failed-check handling with scripted model replies or a real-model run. Failed or unavailable checks never appear as passed.
Earlier behavior	Stage 1 tool, error, action-limit, output-limit, and bug-fix tests still pass.
Feature	Protected feature checks fail on the starting code and pass after the change. Existing callers and regression checks still work.

Scripted replies or real-model runs may verify failure recovery. Core tests must run without a live account or API key. Use a real model for the working feature.

Choose a feature

Continue with your selected repository. Choose a feature that changes at least two interacting modules, preserves existing callers, and handles an error case.

- Example: add optional expiry to stock reservations.
- Release expired reservations so their stock becomes available again.
- Reject invalid expiry values with a clear error.
- Keep existing reservation calls working when no expiry is provided.

For this example, the feature may involve the service, domain rules, and persistence code. Choose an equivalent business feature for your own repository.

The harness must find and change the relevant modules itself. Verify the edit–test–fix loop with scripted replies or a real-model run. A feature that succeeds on the first attempt is acceptable.

Prepare the acceptance check

1. Record the target repository and exact starting commit. Write the feature request, allowed changes, and acceptance commands. Save them with the task.
2. Write feature checks and a regression check. Confirm the feature check fails on the starting code for the expected reason.
3. Keep this acceptance check outside the agent's writable area. Run it against the resulting code in contained execution.
4. Run the harness with a real model. Check the final diff and passing acceptance and regression checks. Identify any manual help.

Delivery 2 checklist

- Runnable source code via repository link and an exact commit, or a versioned ZIP.
- Dependencies, launch commands, and example settings without secrets.
- Automated tests for Stage 2 and the Stage 1 behavior that must still work.
- A short README with setup, model settings, design decisions, known limits, and credits for reused code and AI help; diagrams matching your implementation.
- A video no longer than 4 minutes showing and explaining what you implemented. Choose your own examples and identify any manual help.

Submit individually to the Moodle assignment, as for Part 1. Part 2 receives formative feedback, without a separate grade. Address missing or failing functionality in Part 3; no separate Part 2 resubmission is required. The final project grade uses all three deliveries.